

Document

Quancheng Peng

February 11, 2026

Contents

1	Quick start	1
1.1	Compiling	1
1.2	Execution	1
1.3	Postprocess	1
1.4	Hardware dependency	1
2	Examples	3
2.1	Examples with domain decomposition	3
2.2	Examples without domain decomposition	4
3	Developer documentation	5
3.1	Finite element	5
3.1.1	Isoparametric element	5
3.1.2	Numerical integration	5
3.2	Contact	6
3.2.1	Global contact search	6
3.2.2	Local contact search	6
3.3	Domain decomposition	7
3.4	Linear algebra	7
3.4.1	Nonstationary solver	7
3.4.1.1	Conjugate gradient method	7
3.4.1.2	Chebyshev polynomial	9
3.4.2	Stationary solver	9
3.4.3	Multigrid method	10
3.5	Computer science	10
3.5.1	Hardware	10
3.5.1.1	CPU	10
3.5.1.2	Interface	11
3.5.2	Software	11
3.5.3	Architecture	11
3.6	Miscellaneous	13
3.6.1	Important	13
3.6.2	Moderate	13
3.7	Prospect	13

Chapter 1

Quick start

1.1 Compiling

- Download the source code and extract it, then:

```
cd DDPCA-ADMM-dev-2.0.0
make -j$(nproc)
```

- Only Ubuntu 22.04.5 LTS is tested. Higher versions of Ubuntu should also be compilable, but this has not been verified yet.

1.2 Execution

- Please see the Chapter 2.
- It is recommended to install jemalloc:

```
sudo apt install libjemalloc-dev -y
```

1.3 Postprocess

- Please run the .m scripts in "Postprocess.m" by MATLAB.

1.4 Hardware dependency

Our source code shows high dependency on the hardware. Please make sure that the hardware meets the following requirements:

- In `Utility.hpp`, the `constexpr I64 nfsAlign = 64;` fixes the cahce line size as 64. (Required)
- In `General.hpp`, the `numbThreads = std::thread::hardware_concurrency() / 2;` assumes that SMT (simultaneous multi-threading) is enabled. (Optional)
- In `ThreadManager.cpp`, the `pthread_setaffinity_np(tid, sizeof(cpuset), &cpuset)` binds the thread to the CPU core. (Optional)

Chapter 2

Examples

All examples has been tested on Ubuntu 22.04.5 LTS. Before running the examples, please enter the directory:

```
cd Examples
```

In all the examples, the alias `jem` is used to enable `jemalloc`:

```
alias jem='LD_PRELOAD=/usr/lib/x86_64-linux-gnu/libjemalloc.so.2'
```

In order to obtain higher runtime efficiency, it is recommended to enable transparent huge page by:

```
echo always | sudo tee /sys/kernel/mm/transparent_hugepage/enabled
```

2.1 Examples with domain decomposition

■ TestBeam, on a machine with two AMD EPYC 7742 and 512GB DDR4 RAM.

- Without `jemalloc` and without nested parallelism:

```
./TestBeam
```

- With `jemalloc` and without nested parallelism:

```
jem ./TestBeam
```

- With `jemalloc` and with nested parallelism:

```
jem ./TestBeam 128 4 2
```

`128` is the number of total CPU cores, `4` is the number of threads for per subdomain, and `2` is the number of threads for per interface. The time cost with nested parallelism will be about half of that without nested parallelism.

- The default number of subdomains is 32 by `domaNumb = {16, 2, 1};`. By `domaNumb = {32, 4, 1};`, the number of subdomains is 128.

```
jem ./TestBeam 128 1 1
```

The time cost of setting `128 1 1` is just a little bit lower than that of `128 4 1`: for AMD EPYC 7742, 4 CPU cores share the same 16MB L3 cache. The time cost of setting `64 1 1` is about twice that of `128 1 1`.

- If there is not enough RAM (for example, less than 32GB), please use `Ddpca::I64 globLeve = 1;`.
- If there is not enough CPU cores (for example, less than 32 CPU cores), please do not use nested parallelism.

■ TestBlock, on a machine with two AMD EPYC 7742 and 512GB DDR4 RAM.

```
jem ./TestBeam 128 4 2
```

- TestCylinder, on a machine with two AMD EPYC 7742 and 512GB DDR4 RAM.

```
jem ./TestBeam 128 2 1
```

- TestDehw, on a machine with two AMD EPYC 7742 and 512GB DDR4 RAM.

```
jem ./TestBeam 128 2 1
```

- TestPureTorsion, on a machine with two AMD EPYC 7742 and 512GB DDR4 RAM.

```
jem ./TestBeam 128 4 2
```

2.2 Examples without domain decomposition

- TestNDDBeam, on a machine with two AMD EPYC 7742 and 512GB DDR4 RAM.

- If SMT is enabled:

```
jem ./TestNDDBeam
```

- If SMT is not enabled:

```
jem ./TestNDDBeam 128 1 1
```

- TestNDDPureTorsion, on a machine with two AMD EPYC 7742 and 512GB DDR4 RAM.

- Different to SMP;

Chapter 3

Developer documentation

3.1 Finite element

3.1.1 Isoparametric element

■ Coordinate transformation [1]

$$d\boldsymbol{\xi} = \begin{bmatrix} \frac{\partial x}{\partial \xi} d\xi \\ \frac{\partial y}{\partial \xi} d\xi \\ \frac{\partial z}{\partial \xi} d\xi \end{bmatrix}, d\boldsymbol{\eta} = \begin{bmatrix} \frac{\partial x}{\partial \eta} d\eta \\ \frac{\partial y}{\partial \eta} d\eta \\ \frac{\partial z}{\partial \eta} d\eta \end{bmatrix}, d\boldsymbol{\zeta} = \begin{bmatrix} \frac{\partial x}{\partial \zeta} d\zeta \\ \frac{\partial y}{\partial \zeta} d\zeta \\ \frac{\partial z}{\partial \zeta} d\zeta \end{bmatrix} \quad (3.1)$$

Partial derivatives, Jacobian matrix, gradient tensor

$$\begin{bmatrix} \frac{\partial N_i}{\partial \xi} \\ \frac{\partial N_i}{\partial \eta} \\ \frac{\partial N_i}{\partial \zeta} \end{bmatrix} = \begin{bmatrix} \frac{\partial x}{\partial \xi} & \frac{\partial y}{\partial \xi} & \frac{\partial z}{\partial \xi} \\ \frac{\partial x}{\partial \eta} & \frac{\partial y}{\partial \eta} & \frac{\partial z}{\partial \eta} \\ \frac{\partial x}{\partial \zeta} & \frac{\partial y}{\partial \zeta} & \frac{\partial z}{\partial \zeta} \end{bmatrix} \begin{bmatrix} \frac{\partial N_i}{\partial \xi} \\ \frac{\partial N_i}{\partial \eta} \\ \frac{\partial N_i}{\partial \zeta} \end{bmatrix}, \mathbf{J}^T = \frac{\partial (x, y, z)}{\partial (\xi, \eta, \zeta)} = \nabla_{\boldsymbol{\xi}} \mathbf{x} \quad (3.2)$$

$$\mathbf{J}^T = \begin{bmatrix} \frac{\partial N_1}{\partial \xi} & \frac{\partial N_2}{\partial \xi} & \dots & \frac{\partial N_m}{\partial \xi} \\ \frac{\partial N_1}{\partial \eta} & \frac{\partial N_2}{\partial \eta} & \dots & \frac{\partial N_m}{\partial \eta} \\ \frac{\partial N_1}{\partial \zeta} & \frac{\partial N_2}{\partial \zeta} & \dots & \frac{\partial N_m}{\partial \zeta} \end{bmatrix} \begin{bmatrix} x_1 & y_1 & z_1 \\ x_2 & y_2 & z_2 \\ \vdots & \vdots & \vdots \\ x_m & y_m & z_m \end{bmatrix} \quad (3.3)$$

Infinitesimal volume

$$dV = d\boldsymbol{\xi} \cdot (d\boldsymbol{\eta} \times d\boldsymbol{\zeta}) = |J| d\xi d\eta d\zeta \quad (3.4)$$

Infinitesimal area

$$dA = |d\boldsymbol{\eta} \times d\boldsymbol{\zeta}|_{\xi=\text{const}} = \sqrt{\left(\frac{\partial y}{\partial \eta} \frac{\partial z}{\partial \zeta} - \frac{\partial y}{\partial \zeta} \frac{\partial z}{\partial \eta}\right)^2 + \left(\frac{\partial z}{\partial \eta} \frac{\partial x}{\partial \zeta} - \frac{\partial z}{\partial \zeta} \frac{\partial x}{\partial \eta}\right)^2 + \left(\frac{\partial x}{\partial \eta} \frac{\partial y}{\partial \zeta} - \frac{\partial x}{\partial \zeta} \frac{\partial y}{\partial \eta}\right)^2} d\eta d\zeta \quad (3.5)$$

3.1.2 Numerical integration

■ Gauss Legendre quadrature over a triangle [2]

From Cartesian coordinate to area/barycentric/natural coordinate

$$\begin{cases} x = L_i x_i + L_j x_j + L_m x_m \\ y = L_i y_i + L_j y_j + L_m y_m \end{cases} \quad (3.6)$$

$$dx dy = 2A dL_i dL_j \quad (3.7)$$

where A is the area of the triangle and is constant.

From triangle to standard triangle as shown in Fig. 3.1

$$\begin{cases} L_i = s \\ L_j = t \\ L_m = 1 - s - t \end{cases} \quad (3.8)$$

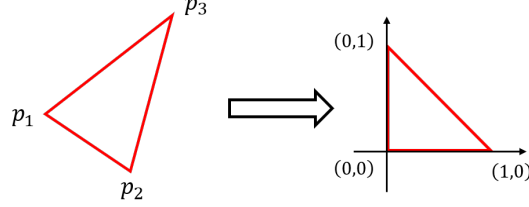


Figure 3.1: From triangle to standard triangle

From standard triangle $\{(x, y) | 0 \leq x, y \leq 1, x + y \leq 1\}$ to standard square $\{(\xi, \eta) | -1 \leq \xi, \eta \leq 1\}$

$$s_k = \frac{1 + \xi_i}{2}, t_k = \frac{(1 - \xi_i)(1 + \eta_j)}{4}, c_k = \frac{(1 - \xi_i)}{8} w_i w_j, 1 \leq k \leq n^2, 1 \leq i, j \leq n \quad (3.9)$$

where ξ_i, η_j and w_i, w_j are Gauss points and weights, respectively.

3.2 Contact

3.2.1 Global contact search

☒ ~~The current bucket sort must satisfy: slave face is smaller than master face.~~

3.2.2 Local contact search

master - non-mortar, slave - mortar;

■ Project slave point onto master face

Point $[\mathbf{r}_s] = \begin{bmatrix} r_{sx} \\ r_{sy} \\ r_{sz} \end{bmatrix}$ is on the slave face(segment), master face is given by

$$\mathbf{r}_m(\xi, \eta) = \sum_{i=1}^4 N_i(\xi, \eta) \mathbf{r}_{mi} = \begin{bmatrix} a_1 + a_2\xi + a_3\eta + a_4\xi\eta \\ b_1 + b_2\xi + b_3\eta + b_4\xi\eta \\ c_1 + c_2\xi + c_3\eta + c_4\xi\eta \end{bmatrix} \quad (3.10)$$

The tangential vector of master face is

$$\frac{\partial \mathbf{r}_m}{\partial \xi} = \begin{bmatrix} a_2 + a_4\eta \\ b_2 + b_4\eta \\ c_2 + c_4\eta \end{bmatrix}, \frac{\partial \mathbf{r}_m}{\partial \eta} = \begin{bmatrix} a_3 + a_4\xi \\ b_3 + b_4\xi \\ c_3 + c_4\xi \end{bmatrix} \quad (3.11)$$

* $\frac{\partial \mathbf{r}_m}{\partial \xi}$ and $\frac{\partial \mathbf{r}_m}{\partial \eta}$ are not perpendicular to each other.

The closest point projection implies

$$\begin{cases} (\mathbf{r}_m(\xi, \eta) - \mathbf{r}_s) \cdot \frac{\partial \mathbf{r}_m(\xi, \eta)}{\partial \xi} = 0 \\ (\mathbf{r}_m(\xi, \eta) - \mathbf{r}_s) \cdot \frac{\partial \mathbf{r}_m(\xi, \eta)}{\partial \eta} = 0 \end{cases} \quad (3.12)$$

After taking Eqs. (3.10) and (3.11) into Eq. (3.12), the following 2-DOF nonlinear system of equations is obtained

$$\begin{cases}
(a_1 a_2 - a_2 r_{sx} + b_1 b_2 - b_2 r_{sy} + c_1 c_2 - c_2 r_{sz}) + (+a_2 a_2 + b_2 b_2 + c_2 c_2) \xi \\
+ (+a_2 a_3 + a_1 a_4 - a_4 r_{sx} + b_2 b_3 + b_1 b_4 - b_4 r_{sy} + c_2 c_3 + c_1 c_4 - c_4 r_{sz}) \eta \\
+ (+2a_2 a_4 + 2b_2 b_4 + 2c_2 c_4) \xi \eta + (+a_3 a_4 + b_3 b_4 + c_3 c_4) \eta^2 + (+a_4 a_4 + b_4 b_4 + c_4 c_4) \xi \eta^2 = 0 \\
(a_1 a_3 - a_3 r_{sx} + b_1 b_3 - b_3 r_{sy} + c_1 c_3 - c_3 r_{sz}) + (+a_3 a_3 + b_3 b_3 + c_3 c_3) \eta \\
+ (+a_2 a_3 + a_1 a_4 - a_4 r_{sx} + b_2 b_3 + b_1 b_4 - b_4 r_{sy} + c_2 c_3 + c_1 c_4 - c_4 r_{sz}) \xi \\
+ (+2a_3 a_4 + 2b_3 b_4 + 2c_3 c_4) \xi \eta + (+a_2 a_4 + b_2 b_4 + c_2 c_4) \xi^2 + (+a_4 a_4 + b_4 b_4 + c_4 c_4) \xi^2 \eta = 0
\end{cases} \quad (3.13)$$

■ Project master point back onto slave face

Point $[\mathbf{r}_m] = \begin{bmatrix} r_{mx} \\ r_{my} \\ r_{mz} \end{bmatrix}$ is on the master face(segment), slave face is given by

$$\mathbf{r}_s(\xi, \eta) = \sum_{i=1}^4 N_i(\xi, \eta) \mathbf{r}_{si} = \begin{bmatrix} a_1 + a_2 \xi + a_3 \eta + a_4 \xi \eta \\ b_1 + b_2 \xi + b_3 \eta + b_4 \xi \eta \\ c_1 + c_2 \xi + c_3 \eta + c_4 \xi \eta \end{bmatrix} \quad (3.14)$$

The tangential vector of master face is

$$\frac{\partial \mathbf{r}_m}{\partial \xi} = \begin{bmatrix} r_{m\xi x} \\ r_{m\xi y} \\ r_{m\xi z} \end{bmatrix}, \quad \frac{\partial \mathbf{r}_m}{\partial \eta} = \begin{bmatrix} r_{m\eta x} \\ r_{m\eta y} \\ r_{m\eta z} \end{bmatrix} \quad (3.15)$$

The closest point projection implies

$$\begin{cases}
(\mathbf{r}_s(\xi, \eta) - \mathbf{r}_m) \cdot \frac{\partial \mathbf{r}_m}{\partial \xi} = 0 \\
(\mathbf{r}_s(\xi, \eta) - \mathbf{r}_m) \cdot \frac{\partial \mathbf{r}_m}{\partial \eta} = 0
\end{cases} \quad (3.16)$$

After taking Eqs. (3.14) and (3.15) into Eq. (3.16), the following 2-DOF nonlinear system of equations is obtained

$$\begin{cases}
\left(a_1 r_{m\xi x} + b_1 r_{m\xi y} + c_1 r_{m\xi z} - r_m \cdot \frac{\partial \mathbf{r}_m}{\partial \xi} \right) + (a_2 r_{m\xi x} + b_2 r_{m\xi y} + c_2 r_{m\xi z}) \xi \\
+ (a_3 r_{m\xi x} + b_3 r_{m\xi y} + c_3 r_{m\xi z}) \eta + (a_4 r_{m\xi x} + b_4 r_{m\xi y} + c_4 r_{m\xi z}) \xi \eta = 0 \\
\left(a_1 r_{m\eta x} + b_1 r_{m\eta y} + c_1 r_{m\eta z} - r_m \cdot \frac{\partial \mathbf{r}_m}{\partial \eta} \right) + (a_2 r_{m\eta x} + b_2 r_{m\eta y} + c_2 r_{m\eta z}) \xi \\
+ (a_3 r_{m\eta x} + b_3 r_{m\eta y} + c_3 r_{m\eta z}) \eta + (a_4 r_{m\eta x} + b_4 r_{m\eta y} + c_4 r_{m\eta z}) \xi \eta = 0
\end{cases} \quad (3.17)$$

■ Computational geometry

- intersect of two 2d line segments;
- calculating the area and centroid of a polygon in 2d [3];

3.3 Domain decomposition

★ [triangulation alignment for both \$\Omega_H\$ and \$\Omega_h\$](#) .

3.4 Linear algebra

3.4.1 Nonstationary solver

3.4.1.1 Conjugate gradient method

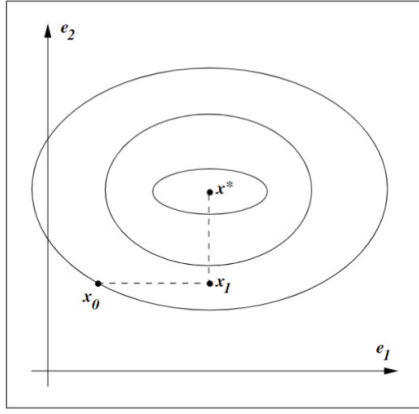
■ Semi-axis alignment [4]:

- matrix $\mathbf{S}^T \mathbf{A} \mathbf{S}$ is diagonal, $\mathbf{S} = [\mathbf{p}_0, \mathbf{p}_1, \dots, \mathbf{p}_{n-1}]$, $\mathbf{p}_i^T \mathbf{A} \mathbf{p}_j = 0, i \neq j$
- coordinate transformation: $\hat{\mathbf{x}} = \mathbf{S}^{-1} \mathbf{x}$, $\frac{1}{2} \mathbf{x} \mathbf{A} \mathbf{x} - \mathbf{b}^T \mathbf{x} = \frac{1}{2} \hat{\mathbf{x}} (\mathbf{S}^T \mathbf{A} \mathbf{S}) \hat{\mathbf{x}} - (\mathbf{S}^T \mathbf{b})^T \hat{\mathbf{x}}$
- converges in n iterations;

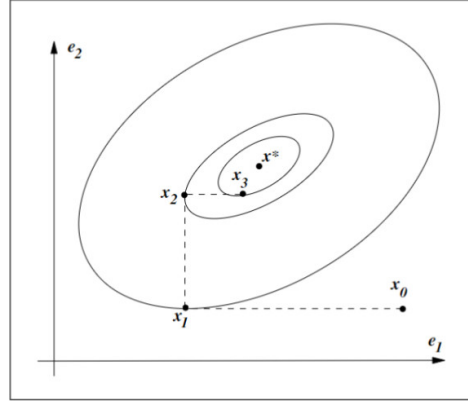
■ Conjugate direction method

- $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k$. The step is determined by “search direction is tangent to isoline”, $\alpha_k = \frac{-\mathbf{r}_k^T \mathbf{p}_k}{\mathbf{p}_k^T \mathbf{A} \mathbf{p}_k}$;

$$\mathbf{r}_{k+1} = \mathbf{r}_k + \alpha_k \mathbf{A} \mathbf{p}_k \quad (3.18)$$



Successive minimizations along the coordinate directions find the minimizer of a quadratic with a diagonal Hessian in n iterations.



Successive minimization along coordinate axes does not find the solution in n iterations, for a general convex quadratic.

Figure 3.2: Semi-axis alignment

- In Fig. 3.2, error $e_k = x_k - x^*$ is conjugate to search direction $e_k^T A p_i = 0, i = 0, 1, \dots, k-1$;
- residual $r_k = Ax_k - b = A e_k$ is orthogonal to search direction;

$$r_k^T p_i = 0, i = 0, 1, \dots, k-1 \quad (3.19)$$

■ Conjugate gradient method [5] Fig. 3.3

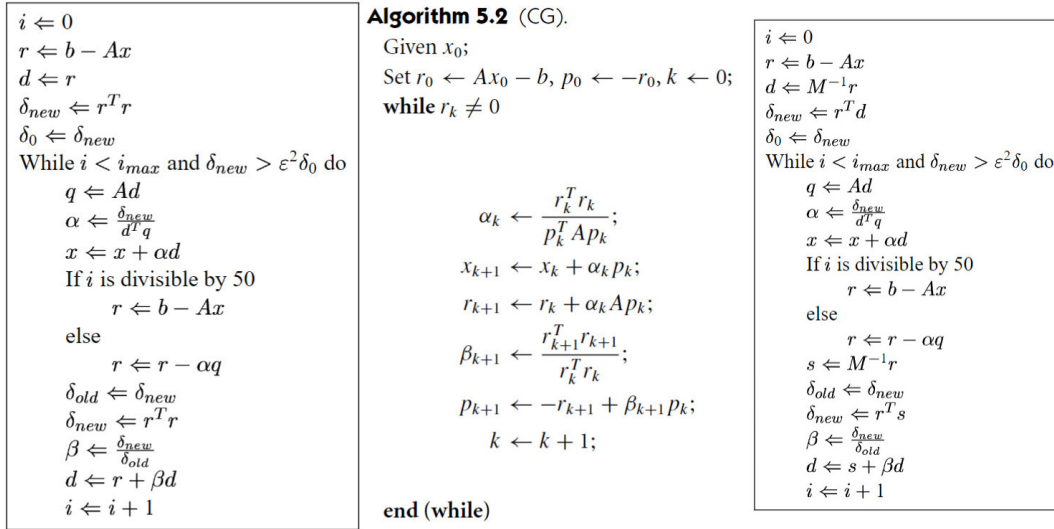


Figure 3.3: Conjugate gradient method

- select search direction by gradient

$$p_0 = r_0, p_k = -r_k + \beta_k p_{k-1} \quad (3.20)$$

- the coefficient is determined by “ p_k and p_{k-1} are conjugate”, $\beta_k = \frac{r_k^T A p_{k-1}}{p_{k-1}^T A p_{k-1}}$;
- From Eqs. (3.18) and (3.20), the Krylov space

$$\text{span}\{r_0, r_1, \dots, r_k\} = \text{span}\{p_0, p_1, \dots, p_k\} = \text{span}\{r_0, A r_0, \dots, A^k r_0\} \quad (3.21)$$

- From Eqs. (3.19) and (3.21), residuals are orthogonal to each other $r_k^T r_i = 0, i = 0, 1, \dots, k-1$;
- Search directions are conjugate to each other $p_k^T A p_i = 0, i = 0, 1, \dots, k-1$. $A p_{i-2} \subset \text{span}\{r_0, r_1, \dots, r_{i-1}\}$
- Simplification: from Eq. (3.18), $\alpha_k = \frac{r_k^T r_k}{p_k^T A p_k}$; from Eqs. (3.18) and (3.20), $\beta_{k+1} = \frac{r_{k+1}^T r_{k+1}}{r_k^T r_k}$

3.4.1.2 Chebyshev polynomial

■ Why Chebyshev polynomial

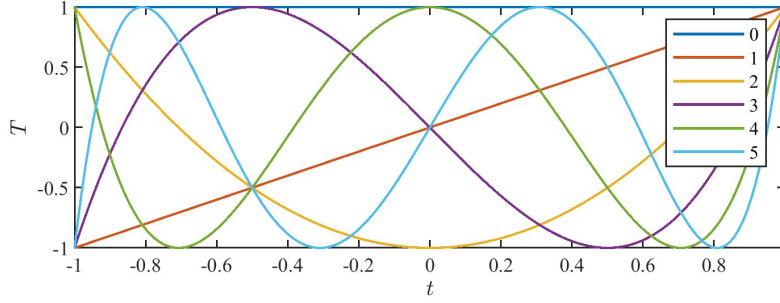


Figure 3.4: Chebyshev polynomial

- Chebyshev polynomial

$$T_0(t) = 1, T_1(t) = t, T_k(t) = 2tT_{k-1}(t) - T_{k-2}(t) \quad (3.22)$$

$$d = \frac{\lambda_{\max} + \lambda_{\min}}{2}, c = \frac{\lambda_{\max} - \lambda_{\min}}{2}, t = \frac{\lambda - d}{c}, 0 < \lambda_{\min} \leq \lambda \leq \lambda_{\max} < +\infty \quad (3.23)$$

$$\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^T, \frac{\mathbf{A} - d\mathbf{I}}{c} = \mathbf{Q}\frac{\mathbf{\Lambda} - d\mathbf{I}}{c}\mathbf{Q}^T, T_k\left(\frac{\mathbf{A} - d\mathbf{I}}{c}\right) = \mathbf{Q}T_k\left(\frac{\mathbf{\Lambda} - d\mathbf{I}}{c}\right)\mathbf{Q}^T \quad (3.24)$$

- The smaller $P(\lambda_i)$ is, the better: Chebyshev polynomial $|T_k(t)| \leq 1, -1 \leq t \leq 1$ is a good choice.

$$\mathbf{e}_0 = \mathbf{x}^* - \mathbf{x}_0 = \mathbf{Q}\mathbf{\xi}_0, P(\mathbf{A})\mathbf{e}_0 = \mathbf{Q}P(\mathbf{\Lambda})\mathbf{Q}^T\mathbf{Q}\mathbf{\xi}_0 = \mathbf{Q}P(\mathbf{\Lambda})\mathbf{\xi}_0 \quad (3.25)$$

■ Chebyshev iteration

$$\begin{cases} \mathbf{r}_k = \mathbf{b} - \mathbf{A}\mathbf{x}_k \\ \mathbf{p}_k = \alpha_k \mathbf{r}_k + \beta_k \mathbf{p}_{k-1}, \mathbf{p}_{-1} = \mathbf{0} \\ \mathbf{x}_{k+1} = \mathbf{x}_k + \mathbf{p}_k \end{cases} \quad (3.26)$$

- From $\mathbf{r}_k = \mathbf{A}\mathbf{e}_k$ and the second equation of Eq. (3.26), search direction

$$\mathbf{p}_k = [\mathbf{I} - P_{k+1}(\mathbf{A})]\mathbf{e}_0 \quad (3.27)$$

- Error update = error multiplied by polynomial

$$\mathbf{e}_{k+1} = \mathbf{x}^* - \mathbf{x}_{k+1} = \mathbf{e}_k - \mathbf{p}_k = P_{k+1}(\mathbf{A})\mathbf{e}_0 \quad (3.28)$$

$$\mathbf{e}_1 = \left(\mathbf{I} - \frac{1}{d}\mathbf{A}\right)\mathbf{e}_0, P_1(\lambda_i) = 1 - \frac{\lambda_i}{d} = -\frac{c}{d}T_1\left(\frac{\lambda_i - d}{c}\right) \quad (3.29)$$

$$\mathbf{e}_2 = \left(\mathbf{I} - \frac{2}{d}\mathbf{A} + \frac{2\mathbf{A}^2}{2d^2 - c^2}\right)\mathbf{e}_0, P_2(\lambda_i) = 1 - \frac{2\lambda_i}{d} + \frac{2\lambda_i^2}{2d^2 - c^2} = T_2\left(\frac{\lambda_i - d}{c}\right) \quad (3.30)$$

- According to that $P_{k+1}(\mathbf{A})$ is Chebyshev polynomial, coefficients α_k and β_k can be determined. The above actually gives a verification for this.

$$\alpha_0 = \frac{1}{d}, \beta_0 = 0; \alpha_1 = \frac{2d}{2d^2 - c^2}, \beta_1 = \alpha_1 d - 1; \alpha_k = \frac{1}{d - \frac{c^2}{4}\alpha_{k-1}}, \beta_k = \alpha_k d - 1 \quad (3.31)$$

3.4.2 Stationary solver

- Matrix partitioning $\mathbf{A} = \mathbf{D} + \mathbf{L} + \mathbf{U}$;

- Jacobian $\mathbf{x}^{(k+1)} = \mathbf{D}^{-1}[\mathbf{b} - (\mathbf{L} + \mathbf{U})\mathbf{x}^{(k)}]$;

- Gauss-Seidel $\mathbf{x}^{(k+1)} = \mathbf{D}^{-1}(\mathbf{b} - \mathbf{L}\mathbf{x}^{(k+1)} - \mathbf{U}\mathbf{x}^{(k)})$;

$$\bullet \mathbf{x}^{(k+1)} = (\mathbf{D} + \mathbf{L})^{-1}(\mathbf{b} - \mathbf{U}\mathbf{x}^{(k)});$$

- Symmetric Gauss-Seidel

- $\mathbf{x}_1 = (\mathbf{D} + \mathbf{L})^{-1} (\mathbf{b} - \mathbf{U}\mathbf{x}_0)$;
 - $\mathbf{x}_2 = (\mathbf{D} + \mathbf{U})^{-1} (\mathbf{b} - \mathbf{L}\mathbf{x}_1) = (\mathbf{D} + \mathbf{U})^{-1} (\mathbf{D}\mathbf{x}_1 + \mathbf{U}\mathbf{x}_0)$ reuses $\mathbf{U}\mathbf{x}_0$;
- Successive over-relaxation (SOR) $\mathbf{x}^{(k+1)} = (1 - \omega) \mathbf{x}^{(k)} + \omega \mathbf{D}^{-1} (\mathbf{b} - \mathbf{L}\mathbf{x}^{(k+1)} - \mathbf{U}\mathbf{x}^{(k)})$;
- $\mathbf{x}^{(k+1)} = (\mathbf{D} + \omega \mathbf{L})^{-1} \left\{ [(1 - \omega) \mathbf{D} - \omega \mathbf{U}] \mathbf{x}^{(k)} + \omega \mathbf{b} \right\}$;
- Symmetric SOR (SSOR)
- $\mathbf{x}_1 = (\mathbf{D} + \omega \mathbf{L})^{-1} [(1 - \omega) \mathbf{D} - \omega \mathbf{U}] \mathbf{x}_0 + \omega (\mathbf{D} + \omega \mathbf{L})^{-1} \mathbf{b} = (\mathbf{D} + \omega \mathbf{L})^{-1} (\mathbf{p}_0 + \omega \mathbf{b})$;
 - $\mathbf{x}_2 = (\mathbf{D} + \omega \mathbf{U})^{-1} [(1 - \omega) \mathbf{D} - \omega \mathbf{L}] \mathbf{x}_1 + \omega (\mathbf{D} + \omega \mathbf{U})^{-1} \mathbf{b} = (\mathbf{D} + \omega \mathbf{U})^{-1} [(2 - \omega) \mathbf{D}\mathbf{x}_1 - \mathbf{p}_0]$;

3.4.3 Multigrid method

Multigrid cycle $u_k^{m+1} = \text{MGCYC}(k, \gamma, u_k^m, L_k, f_k, v_1, v_2)$

(1) Presmoothing

- Compute $\bar{u}_k^m$ by applying $v_1 (\geq 0)$ smoothing steps to u_k^m

$$\bar{u}_k^m = \text{SMOOTH}^{v_1}(u_k^m, L_k, f_k).$$

(2) Coarse grid correction

- Compute the defect $\bar{d}_k^m = f_k - L_k \bar{u}_k^m$.
- Restrict the defect $\bar{d}_{k-1}^m = I_k^{k-1} \bar{d}_k^m$.
- Compute an approximate solution $\hat{v}_{k-1}^m$ of the defect equation on Ω_{k-1}

$$L_{k-1} \hat{v}_{k-1}^m = \bar{d}_{k-1}^m \quad (2.4.3)$$

by

If $k = 1$, use a direct or fast iterative solver for (2.4.3).
 If $k > 1$, solve (2.4.3) approximately by performing $\gamma (\geq 1)$ k -grid cycles using the zero grid function as a first approximation

$$\hat{v}_{k-1}^m = \text{MGCYC}^\gamma(k-1, \gamma, 0, L_{k-1}, \bar{d}_{k-1}^m, v_1, v_2). \quad (2.4.4)$$

- Interpolate the correction $\hat{v}_k^m = I_{k-1}^k \hat{v}_{k-1}^m$.
- Compute the corrected approximation on Ω_k $u_k^{m, \text{after CGC}} = \bar{u}_k^m + \hat{v}_k^m$.

(3) Postsmoothing

- Compute u_k^{m+1} by applying $v_2 (\geq 0)$ smoothing steps to $u_k^{m, \text{after CGC}}$:

$$u_k^{m+1} = \text{SMOOTH}^{v_2}(u_k^{m, \text{after CGC}}, L_k, f_k).$$

Figure 3.5: Multigrid method [6]

3.5 Computer science

3.5.1 Hardware

3.5.1.1 CPU

■ Caching

- Every CPU core has its own L1/L2 caching, all CPU cores share L3 caching;
- TLB: from virtual address to real physical address. Every CPU core has its own L1/L2 DTLB, all CPU cores share L3 DTLB;

■ NUMA (Non-Uniform Memory Access)

- Different to SMP;

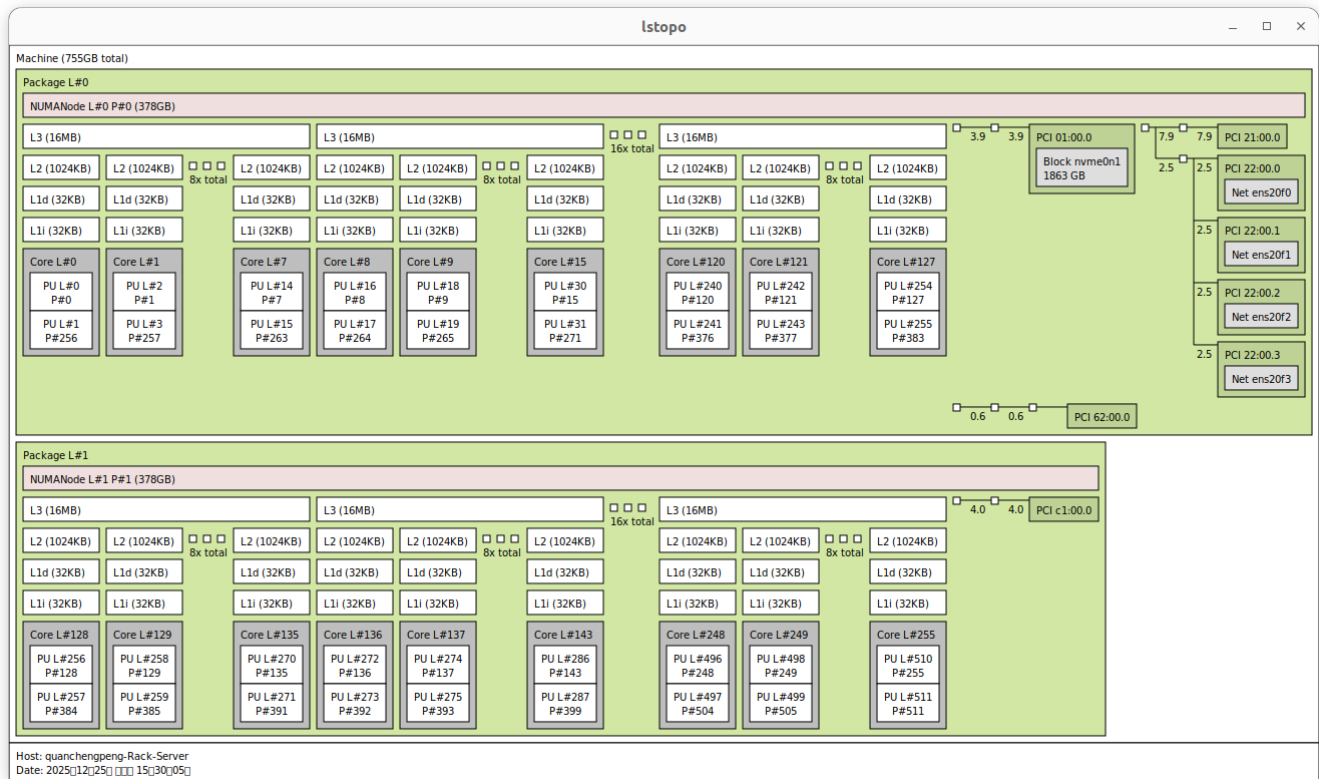


Figure 3.6: Istopo

- Every NUMA node has its own CPU cores and local memory;
- For a CPU core in NUMA node 1, the latency to access local memory is much lower than that to access the remote memory (local memory of NUMA node 2);

3.5.1.2 Interface

■ PCIe Fig. 3.7

- PCIe x1, x4, x8, x16, 1.0, 2.0, 3.0, 4.0, 5.0;
- PCIe Mini SAS HD;

■ Hard disk

- U.2 (NVME PCIe), M.2 (NVME PCIe), SATA Fig. 3.8;

3.5.2 Software

■ Plain old data

■ `semaphore` is much more stable than `condition_variable`;

■ Even if the output is right, we can not say that the code is right: there must be always bugs!

3.5.3 Architecture

■ Distributed parallelism

- Master node, worker node: the same operating system (all Ubuntu 22.04.5 LTS), the same LAN, the same hostname resolution /etc/hosts, the same username;
- SSH no password;
- hostfile on master node; Slurm -with-slurm;

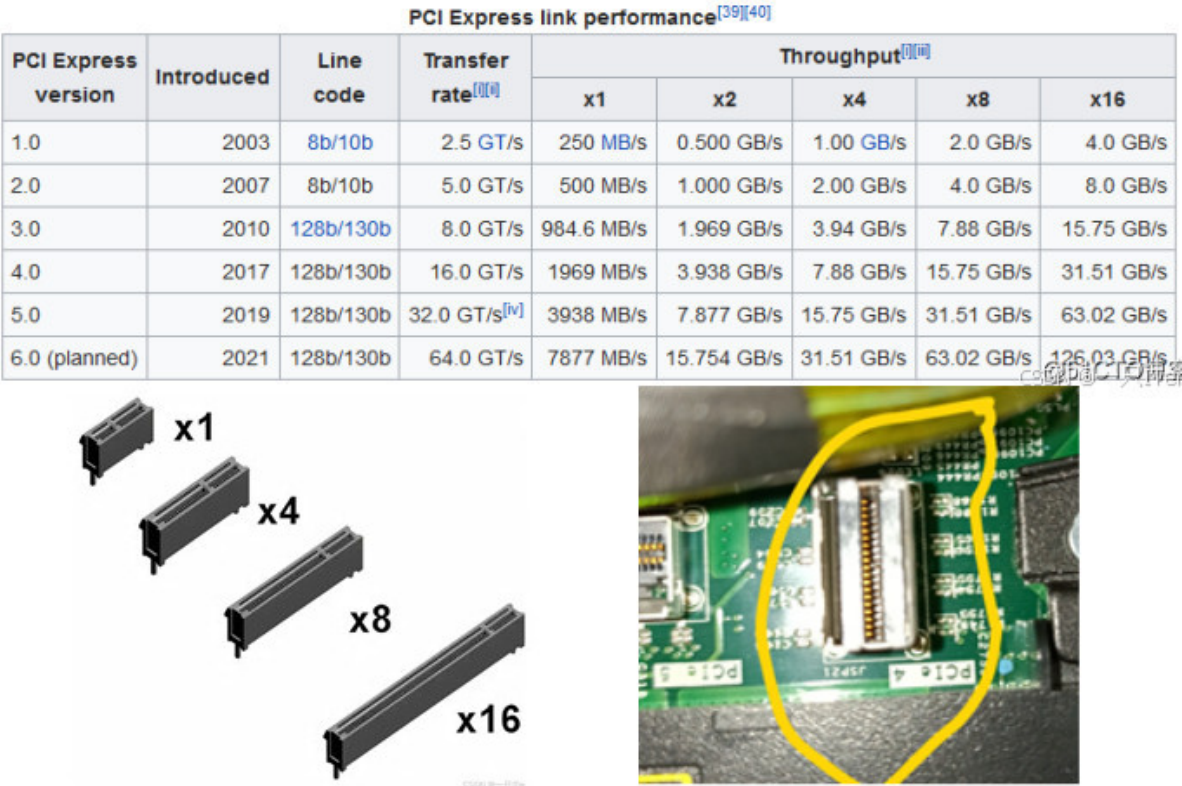


Figure 3.7: PCIe

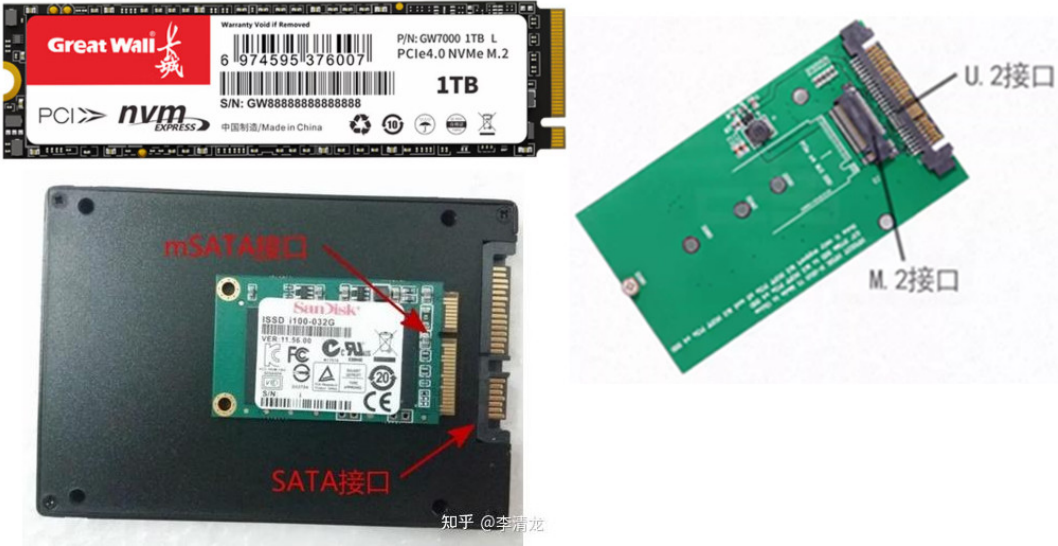


Figure 3.8: M.2, U.2, SATA

3.6 Miscellaneous

3.6.1 Important

- `GeometricMultigrid.cpp`: the level difference of two adjacent elements ≤ 1 ;

3.6.2 Moderate

- Thread management parameters
 - Preferably, `numbThreads` is divisible by `threadsPerDomain/threadsPerInterface`;
 - Total number of levels: `maxLevel + 2, 0 ~ maxLevel + 1`;
 - By default, the AMD SMT is enabled;
- `Log` is not thread safe: better performance;
- `pthread.h`: only support Ubuntu;
- At most two levels of nested parallelism;
- The nested parallelism in `GeometricMultigrid.cpp` is much more complicated!!!!

3.7 Prospect

- add RAM, see can it less than 100s: very sure it can when the 24 memory slots are all used!
- learn distributed parallelism with openMPI: try the scnet;
- why abandon "nested parallelism":
 - pursuit for scalability: one CPU core for one domain will be the best choice (or at least a closing choice); Otherwise, how to explain the reason that nested parallelism is better than "one CPU core for one domain";
 - openMPI will be much better in order to avoid access conflict of L3 cache;
- Self-establish is too expensive: i3-12100f 4C 3.4GHz, $500 * 256 = 128000$; ddr5 16G, $500 * 256 = 128000$;
- Self-establish: E8400 2C 3.0GHz+G45 motherboard, $161 * 128 = 20608$; ddr3 8G, $100 * 128 = 12800$; 128G SSD, $90 * 128 = 11520$;
- What is the difference: interpolation of displacement and then projected onto normal direction; interpolation of normal displacement;

Bibliography

- [1] Xucheng Wang. Finite Element Method. Tsinghua University Press, 2003.
- [2] Rathod HT, Nagaraja KV, VENKATESUDU B, and RAMESH NL. Gauss legendre quadrature over a triangle. Journal of the Indian Institute of Science, 2004.
- [3] Nurnberg R. Calculating the area and centroid of a polygon in 2d. www.ma.ic.ac.uk/~rn/centroid.pdf. Accessed: 2013.
- [4] Wright SJ Nocedal J. Numerical Optimization. Springer, 2006.
- [5] Shewchuk JR. An introduction to the conjugate gradient method without the agonizing pain. <https://people.eecs.berkeley.edu/~jrs/papers/cg.pdf>. Accessed: 2026.
- [6] Schuller A Trottenberg U, Oosterlee C. Multigrid. Academic Press, 2001.

